Zara products are ready in your workspace

Today you removed Koton, Viled and Qazaq Republic, because their clothes did not match each other. Good call. **One brand** matches better: Zara is one style. Your workspace now has a saved Zara catalog with real names, ₸ prices, colours, photos and links. Your page can make **Zara-only outfits** from it, and it never contacts Zara by itself.

Already in your workspace (27 Oct 2026)

File	What is in it
`KNOWLEDGE/zara-live.json`	women: 160 items, 40 each: верх, низ, слой, платье
`KNOWLEDGE/zara-live-men.json`	men: 120 items, 40 each: верх, низ, слой
`zara/zara_search.py`	searches Zara and saves the products (code: pages 6–8)
`zara/make_catalog.py`	makes the 2 catalog files (code: pages 9–10)

Made from 7 searches (women: t shirt, jeans, jacket, dress; men: t shirt, jeans, jacket). Each search found 216 products; Zara did not block any search. Every item has a name, a ₸ price, a colour, a photo link and a product link.

Your 3 steps

A. Zara-only outfits
one prompt to your agent (page 2)

›

B. Your rules
one line in AGENTS.md (page 2)

›

C. Fresh prices
once a week, on your laptop (pages 3–4)

The rules (read them first):

1. Your page and your agent **never** fetch zara.com. They only read the saved files.
2. Refresh by hand, on your laptop: **at most 10 searches a day**, **1 minute** between them, never in a loop.
3. Zara shows a check page ("are you a robot?"), a 403 error or an empty page? **Stop.** Do not try tricks to get past it. Keep the old file for that week.
4. Your page shows the name, price, photo and the **link to Zara**. People buy on zara.com. Do not copy or sell Zara's catalog.

workspace A. Zara-only outfits (OpenCode)

20–30 min

Your page now makes outfits from Pinterest trends, as text. Keep that. Add one switch: **“Zara”**. With the switch on, every outfit card shows real Zara items (photo, name, ₸ price, link) that fit the trend, the palette and the weather.

```
/plan In projects/virtual-stylist: add an optional "Zara-only" mode to the outfits.
Keep the Pinterest trend outfits as they are. Add a switch "Показать вещи Zara" next
to the outfit button. When it is on, read ONLY the saved file KNOWLEDGE/zara-live.json
(женский) or KNOWLEDGE/zara-live-men.json (мужской) with fetch (relative path, no
server change), and give the model the list of Zara items (id, name, category, colour,
price) with my palette, the weather and the Pinterest trends. Each outfit card: 1 top
+ 1 bottom + an optional layer, or a dress for women, all from Zara, with photo, name,
₸ price and a link to zara.com; show "Zara KZ · checked " under the cards. Use only
ids that are in the file. Never fetch zara.com. If the file is missing, hide the
switch. Show me the plan first.
```

✓ Done when: with the switch on, each card shows 2–3 real Zara photos with prices and working links, and the colours match your palette.

workspace B. Update your rules (AGENTS.md)

5 min

Your `AGENTS.md` says "Do not scrape Zara KZ automatically" and "Never bypass the check". These rules stay true. Add only where the Zara data comes from.

```
/plan In projects/virtual-stylist/AGENTS.md, under Limits, keep the Zara rule and add:
"Zara KZ (mentor's OK, 27 Oct 2026): the page reads only the saved files
KNOWLEDGE/zara-live.json and KNOWLEDGE/zara-live-men.json, made by hand with
zara/zara_search.py + zara/make_catalog.py on a laptop (at most 10 searches a day;
stop at any bot check). The page, the server and the agent never fetch zara.com.
Refresh about once a week." Show me the change first.
```

✓ Done when: AGENTS.md has the new line, and the old "do not scrape" rule is still there.

C. Fresh prices: once a week, on your laptop

Prices and items change. About once a week, make new files on your laptop. Why the laptop? The script needs a full Chromium browser; your class workspace has none and cannot install one.

laptop 1. Get the code and install (once)

10 min

In your workspace Explorer, open `projects/virtual-stylist/zara`, right-click `zara_search.py` → **Download**, the same for `make_catalog.py`. Put both in one folder, for example `Documents/zara`. You need Python 3.9 or newer (python.org). Open a terminal in that folder (Windows: PowerShell; Mac: Terminal):

```
pip install playwright
python -m playwright install chromium
```

Mac: if `pip` or `python` is not found, use `pip3` and `python3`.

✓ **Done when: the second command ends without an error.**

laptop 2. Search Zara

about 2 min per search

A Chromium window opens, scrolls for about 15 seconds and closes. Do not touch it. The phrase is always `<item> from Zara kz`. Women is the default; add `--section MAN` for men. **Rename the result after every search**, then wait 1 minute.

```
python zara_search.py "t shirt from Zara kz"                    → rename
zara_products.json to tshirt_w.json
python zara_search.py "jeans from Zara kz"                      → jeans_w.json
python zara_search.py "jacket from Zara kz"                     → jacket_w.json
python zara_search.py "dress from Zara kz"                      → dress_w.json
python zara_search.py "t shirt from Zara kz" --section MAN      → tshirt_m.json
python zara_search.py "jeans from Zara kz" --section MAN        → jeans_m.json
python zara_search.py "jacket from Zara kz" --section MAN       → jacket_m.json
```

```
Search page: https://www.zara.com/kz/ru/search?searchTerm=t%20shirt&section=WOMAN
JSON responses: 27, products: 216
- ФУТБОЛКА С ПРИНТОМ I LOVE NY® | 13990.0 | WOMAN | https://www.zara.com/kz/ru/...
```

✓ **Done when: you have 7 files, and each search printed "products" more than 0.**

laptop 3. Make the 2 catalog files

1 min

One file for women, one for men, in your shop-catalog format. The script finds the category (верх, низ, платье, слой, аксессуар) from Zara's product type, skips shoes, writes names in normal case, and keeps at most 40 items per category.

```
python make_catalog.py women zara-live.json tshirt_w.json jeans_w.json jacket_w.json
dress_w.json
python make_catalog.py men zara-live-men.json tshirt_m.json jeans_m.json jacket_m.json
```

```
zara-live.json: 160 products {'верх': 40, 'низ': 40, 'слой': 40, 'платье': 40},
skipped 1
zara-live-men.json: 120 products {'верх': 40, 'низ': 40, 'слой': 40}, skipped 0
```

✓ **Done when: both files exist, with the categories you need.**

workspace 4. Upload them (they replace the old ones)

2 min

In the Explorer, open `projects/virtual-stylist/KNOWLEDGE` and drag `zara-live.json` and `zara-live-men.json` from your laptop onto it. Say yes to "replace". The `checked` date in the file is the new date.

✓ **Done when: your page shows "Zara KZ · checked" with today's date.**

How a search works, in short: your laptop opens `zara.com/kz/ru/search?searchTerm=…` like a person. The page loads its products from Zara's own data service, 36 at a time. The script listens, keeps the products, and writes the name, the price in ₸, the colour, the photo link and the product link. The full code with explanations is on pages 6–10.

If something goes wrong

You see	Do this
"products: 0"	Zara changed its data. Open `zara_raw.json`, search for a product name, and ask your agent: "*the products are inside this JSON; change find_products and summarize in zara_search.py to read them*".
A check page, 403, or an empty page	**Stop for today.** Use the manual importer. Do not add "stealth" tools.
Too few products	Normal for rare items. More scrolling: change `range(5)` to `range(8)` in the script.
Only women's items	Add `--section MAN` for men. `--section KID` was not tested.
"Executable doesn't exist" (browser)	Run `python -m playwright install chromium` again.
A product is in the wrong category	Add its word to the list `CATEGORY` in `make_catalog.py`, then run step 3 again.

What the script does, in 5 lines

1. Reads your phrase: `t shirt from Zara kz` → item `t shirt`, country `kz`.
2. Opens `zara.com/kz/ru/search?searchTerm=t%20shirt&section=WOMAN` in Chromium, like a person, and scrolls 5 times.
3. The page loads its products from Zara's own API (`/api/storefront/2/stores/11749/grids/search`, 36 per page). The script saves these answers.
4. It finds each product: `name`, price `pricing.price.current.value` $\times 10^{-2}$ = ₸, colour, photo `media[].asset.url`, link from `seo`.
5. It writes `zara_products.json`. No computer use, no login, no personal data.

For your Demo Day talk: "Our stylist uses real products from 4 shops: Koton, Viled, Qazaq Republic and Zara. Zara blocks bots, so we save its catalog by hand once a week and the app only reads the saved file."

THE CODE · ZARA_SEARCH.PY (1 OF 3)

Top: what the file is, the imports, the phrase rule

lines 1–25

The text in """ """ is only a note. `sync_playwright` starts and controls Chromium. `PHRASE` is a pattern: "*anything* from zara *2 letters*".

```
"""Search a Zara country store and save the products as JSON.

Usage:
    pip install playwright && python -m playwright install chromium
    python zara_search.py "t shirt from Zara kz"                  (women's section)
    python zara_search.py "t shirt from Zara kz" --section MAN    (men's section)

The script opens the Zara search page in a real Chromium browser, because
Zara blocks plain HTTP clients. It records the JSON responses that the page
gets from its own API, then finds the product objects in them.

Output:
    zara_raw.json       all JSON responses (use it to inspect the real schema)
    zara_products.json  name, price, URL and image for each product found
"""

import json
import re
import sys
from urllib.parse import quote

from playwright.sync_api import sync_playwright

PHRASE = re.compile(r"(?P<q>.+?)\s+from\s+zara\s+(?P<cc>[a-z]{2})\b", re.I)

```

parse(): the phrase → item + country

lines 26–32

`"t shirt from Zara kz"` gives `("t shirt", "kz")`. Another form of phrase gives a clear error.

```

def parse(text):
    m = PHRASE.match(text.strip())
    if not m:
        raise ValueError('Use the format: "<item> from Zara <country code>"')
    return m["q"].strip(), m["cc"].lower()

```

find_products(): find every product in Zara's answers

lines 33–55

Zara's answer is a big tree of data. This function walks through all of it and keeps every object that has `identifier.id`, `name` and `pricing`: that is one product. Each id is kept once.

```

def find_products(node, found=None):
    """Walk any JSON tree and collect objects that look like products.

    Zara (checked 27 Oct 2026, KZ store 11749): the search API
    /api/storefront/2/stores/<store>/grids/search sends products in "components",
    each with identifier.id, name and pricing.price.current. The older format
    (id + name + price on one object) is still accepted."""
    if found is None:
        found = {}
    if isinstance(node, dict):
        ident = node.get("identifier")
        if isinstance(ident, dict) and "id" in ident and "name" in node and "pricing" in node:
            found.setdefault(ident["id"], node)
        elif "name" in node and "price" in node and "id" in node:
            found.setdefault(node["id"], node)
        for value in node.values():
            find_products(value, found)
    elif isinstance(node, list):
        for value in node:
            find_products(value, found)
    return found

```

THE CODE · ZARA_SEARCH.PY (2 OF 3)

first_image(): the photo link

lines 56–70

Looks inside the product for the first link to `static.zara.net` (Zara's photo server) and asks for a photo 750 pixels wide.

```
56
57  def first_image(product):
58      """Return the first image URL in a product object, if one exists."""
59      stack = [product]
60      while stack:
61          node = stack.pop()
62          if isinstance(node, dict):
63              url = node.get("url")
64              if isinstance(url, str) and "static.zara.net" in url:
65                  return url.replace("{width}", "750")
66              stack.extend(node.values())
67          elif isinstance(node, list):
68              stack.extend(reversed(node))
69      return None
70
```

price_of(): the price in tenge

lines 71–80

Zara sends `1399000` with exponent `-2` : 1399000 × 10^{-2} = **13,990 ₸**.

```
71
72  def price_of(product):
73      """Price in normal units (tenge). New format: value * 10**exponent; old format: minor units / 100."""
74      cur = ((product.get("pricing") or {}).get("price") or {}).get("current")
75      if isinstance(cur, dict) and isinstance(cur.get("value"), (int, float)):
76          exp = (cur.get("currency") or {}).get("exponent", -2)
77          return cur["value"] * 10 ** exp, (cur.get("currency") or {}).get("code")
78      price = product.get("price")
79      return (price / 100 if isinstance(price, (int, float)) else price), None
80
```

summarize(): one clean row per product

lines 81–105

Takes only what your app needs: id, name, price, currency, colour, hex colour, Zara's product type (`family`), section, the product link (built from `seo`) and the photo.

```
81
82  def summarize(product, cc, lang):
83      seo = product.get("seo") or {}
84      url = None
85      if seo.get("keyword") and seo.get("seoProductId"):
86          # keyword is already URL-encoded and ends with "-": Zara's own links look like ...i-love-ny--p03366100.html
87          url = f"https://www.zara.com/{cc}/{lang}/{seo['keyword']}-p{seo['seoProductId']}.html"
88          if seo.get("discernProductId"):
89              url += f"?v1={seo['discernProductId']}"
90      price, currency = price_of(product)
91      ident = product.get("identifier") or {}
92      return {
93          "id": ident.get("id", product.get("id")),
94          "name": product.get("name"),
95          "price": price,
96          "currency": currency,
97          "color": (product.get("color") or {}).get("name"),
98          "hex": (product.get("color") or {}).get("hexCode"),
99          "family": ((product.get("classification") or {}).get("family") or {}).get("name"),
100         "section": ((product.get("classification") or {}).get("section") or {}).get("name"),
101         "url": url,
102         "image": first_image(product),
103     }
104
105
```

THE CODE · ZARA_SEARCH.PY (3 OF 3)

zara_search(): open the page in Chromium and listen

lines 106–136

Builds the search link, opens it in a real Chromium window, and saves every JSON answer the page gets. Then it scrolls 5 times (more products load), closes the browser, and runs find_products + summarize.

```
106 def zara_search(text, lang="ru", headless=False, section="WOMAN"):
107     query, cc = parse(text)
108     url = f"https://www.zara.com/{cc}/{lang}/search?searchTerm={quote(query)}&section={section}"
109     payloads = []
110 
111     def on_response(resp):
112         if "json" not in resp.headers.get("content-type", ""):
113             return
114         try:
115             payloads.append({"url": resp.url, "data": resp.json()})
116         except Exception:
117             pass
118 
119     with sync_playwright() as p:
120         browser = p.chromium.launch(headless=headless)
121         page = browser.new_page(locale=f"{lang}-{cc.upper()}")
122         page.on("response", on_response)
123         page.goto(url, wait_until="domcontentloaded")
124         page.wait_for_timeout(5000)
125         for _ in range(5):  # scroll to load more results
126             page.mouse.wheel(0, 4000)
127             page.wait_for_timeout(1500)
128         browser.close()
129 
130     found = {}
131     for item in payloads:
132         find_products(item["data"], found)
133     products = [summarize(p, cc, lang) for p in found.values()]
134     return url, payloads, products
135 
136 
```

The start: read the command, write the 2 files

lines 137–153

Reads your phrase and `--section MAN` if you typed it, runs the search, and writes `zara_raw.json` (everything) and `zara_products.json` (the clean list).

```
137 if __name__ == "__main__":
138     args = sys.argv[1:]
139     section = "WOMAN"
140     if "--section" in args:  # WOMAN (default), MAN or KID
141         i = args.index("--section")
142         section = args[i + 1].upper()
143         del args[i:i + 2]
144     phrase = " ".join(args) or "t shirt from Zara kz"
145     url, payloads, products = zara_search(phrase, section=section)
146     with open("zara_raw.json", "w", encoding="utf-8") as f:
147         json.dump(payloads, f, ensure_ascii=False, indent=2)
148     with open("zara_products.json", "w", encoding="utf-8") as f:
149         json.dump(products, f, ensure_ascii=False, indent=2)
150     print(f"Search page: {url}")
151     print(f"JSON responses: {len(payloads)}, products: {len(products)}")
152     for p in products[:20]:
153         print(f"- {p['name']} | {p['price']} | {p['section']} | {p['url']}")
```

THE CODE · MAKE_CATALOG.PY (1 OF 2)

make_catalog.py: top, the category words and the 40 limit

lines 1–32

Each category has a list of word starts from Zara's product types (Russian store); the first match wins. Shoe words are skipped (your app has no shoes slot). `MAX_PER_CATEGORY = 40` keeps the file small and varied.

```
"""Turn zara_products.json files into a virtual-stylist catalog (same format as KNOWLEDGE/koton-live.json).

Usage (on your laptop, after zara_search.py):
    python make_catalog.py women zara-live.json  tops_w.json jeans_w.json jackets_w.json
    python make_catalog.py men   zara-live-men.json  tops_m.json jeans_m.json

Each input is one zara_products.json that you renamed after a search. The output has
shop, audience, checked (today), source, products (id, name, url, image, price, old_price, color, hex,
stock, category), errors and stale, like the Koton and Qazaq Republic files. Category comes from the
Zara product family: верх, низ, платье, слой, аксессуар. Shoes and unknown families are skipped and
counted in "errors" (the app has no shoes slot).
"""
import datetime
import json
import sys
import time

CATEGORY = [  # first match wins; words from Zara's family names (Russian store)
    ("слой", ["КУРТК", "ПАЛЬТО", "ПИДЖАК", "ЖАКЕТ", "БЛЕЙЗЕР", "ПУХОВИК", "ПАРК", "ТРЕНЧ", "КАРДИГАН", "ЖИЛЕТ",
              "БОМБЕР", "ВЕТРОВК", "ПЛАЩ", "ДУБЛЕНК", "ВЕРХНЯЯ"]),
    ("платье", ["ПЛАТЬ", "САРАФАН", "КОМБИНЕЗОН"]),
    ("низ", ["БРЮК", "ДЖИНС", "ЮБК", "ШОРТ", "ЛЕГИНС", "ЛОСИН"]),
    ("верх", ["ФУТБОЛК", "РУБАШК", "БЛУЗ", "ТОП", "СВИТЕР", "ДЖЕМПЕР", "ПОЛО", "ТОЛСТОВК", "ЛОНГСЛИВ", "СВИТШОТ",
              "ВОДОЛАЗК", "ХУДИ", "МАЙК", "КОРСЕТ", "БОДИ", "ТРИКОТАЖ"]),
    ("аксессуар", ["СУМК", "РЕМ", "ШАРФ", "ШАПК", "ПЛАТОК", "ПЕРЧАТК", "ОЧК", "УКРАШЕН", "СЕРЬГ", "ОЖЕРЕЛ", "БРАСЛЕТ"]),
]
SHOES = ["ОБУВ", "КРОССОВК", "БОТИН", "САПОГ", "ТУФЛ", "КЕД", "САНДАЛ", "ЛОФЕР", "МОКАСИН", "ШЛЕПАНЦ"]


MAX_PER_CATEGORY = 40  # keep Koton, Viled and Qazaq visible: the page picks at random from all shops together
```

THE CODE · MAKE_CATALOG.PY (2 OF 2)

nice_name() and category_of(): clean name, card slot

lines 33-49

`nice_name` : “ФУТБОЛКА С ПРИНТОМ I LOVE NY®” → “Футболка с принтом I LOVE NY®”. `category_of` : “ФУТБОЛКА” → **верх**, “ДЖИНСЫ” → **низ**, “КУРТКА” → **слой**.

```
33  def nice_name(name):
34      """ФУТБОЛКА С ПРИНТОМ I LOVE NY® → Футболка с принтом I LOVE NY® (Russian words in lower case, Latin words kept)."""
35      words = [w.lower() if any("A" ⩽ ch ⩽ "я" or ch in "Ёё" for ch in w) else w for w in (name or "").split()]
36      text = " ".join(words)
37      return text[:1].upper() + text[1:]
38
39
40  def category_of(item):
41      text = f"{item.get('family') or ''} {item.get('name') or ''}".upper()
42      if any(w in text for w in SHOES):
43          return None
44      for cat, words in CATEGORY:
45          if any(w in text for w in words):
46              return cat
47      return None
48
49
```

main(): the catalog file

lines 50-82

Reads your saved search files, keeps products with a link, photo and price, removes doubles, stops at 40 per category, and writes the file in your catalog format (shop, audience, checked date, products).

```
50  def main(audience, out, inputs):
51      products, errors, seen = [], [], set()
52      for path in inputs:
53          for item in json.load(open(path, encoding="utf-8")):
54              if not item.get("url") or not item.get("image") or not isinstance(item.get("price"), (int, float)):
55                  errors.append(f"{path}: no url, image or price: {item.get('name')}")
56                  continue
57              if item["id"] in seen:
58                  continue
59              cat = category_of(item)
60              if cat is None:
61                  errors.append(f"{path}: skipped (shoes or unknown family): {item.get('family')} / {item.get('name')}")
62                  continue
63              if sum(1 for p in products if p["category"] == cat) ⩾ MAX_PER_CATEGORY:
64                  continue
65              seen.add(item["id"])
66              products.append({"shop": "Zara KZ", "id": str(item["id"]), "name": nice_name(item["name"]), "url": item["url"], "ima
67                               "price": int(round(item["price"])), "old_price": None, "color": (item.get("color") or "").lower(),
68                               "hex": item.get("hex"), "stock": None, "category": cat})
69      catalog = {"shop": "Zara KZ", "audience": audience, "checked": datetime.date.today().isoformat(),
70                 "fetched_at": int(time.time()), "source": "https://www.zara.com/kz/ru/ (saved by hand with zara_search.py)",
71                 "products": products, "errors": errors, "stale": False}
72      json.dump(catalog, open(out, "w", encoding="utf-8"), ensure_ascii=False, indent=1)
73      by_cat = {}
74      for p in products:
75          by_cat[p["category"]] = by_cat.get(p["category"], 0) + 1
76      print(f"{out}: {len(products)} products {by_cat}, skipped {len(errors)}")
77
78
79  if __name__ == "__main__":
80      if len(sys.argv) < 4 or sys.argv[1] not in ("women", "men"):
81          sys.exit("usage: python make_catalog.py women|men OUT.json IN1.json [IN2.json ...]")
82      main(sys.argv[1], sys.argv[2], sys.argv[3:])
```